今天的内容

- 微架构—多周期CPU设计

今天的内容

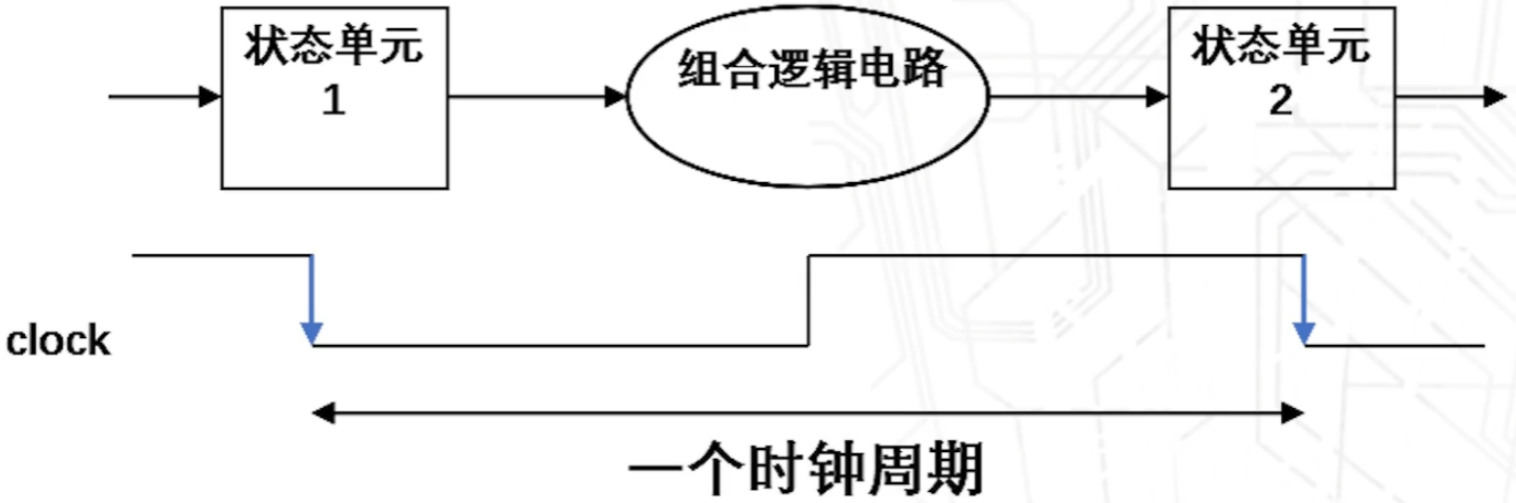
- 微架构—多周期CPU设计

回顾：

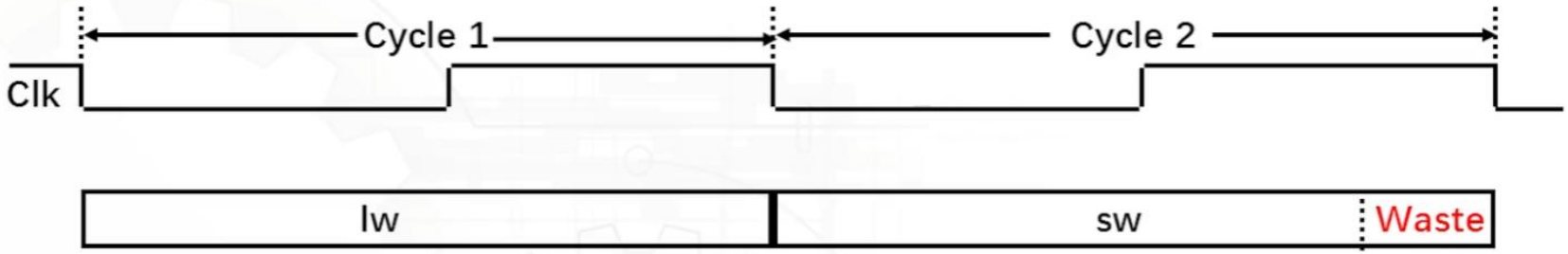
- **微架构介绍：单周期微架构**
  - 单周期 vs. 多周期
  - 指令处理“周期”
  - 数据通路 vs. 控制逻辑
  - 硬件控制 vs. 微程序控制
  - 性能分析：执行时间方程
  - 功率分析：动态功率方程 动态功耗 = 活动因子 × 电容 × 电压平方 × 频率
- **单周期MIPS实现的详细讲解**
  - 数据通路
  - 控制逻辑
  - 关键路径分析
- **（微）架构设计原则**

回顾：单周期处理器

单周期处理器：一个时钟周期完成一条指令



回顾：单周期处理器

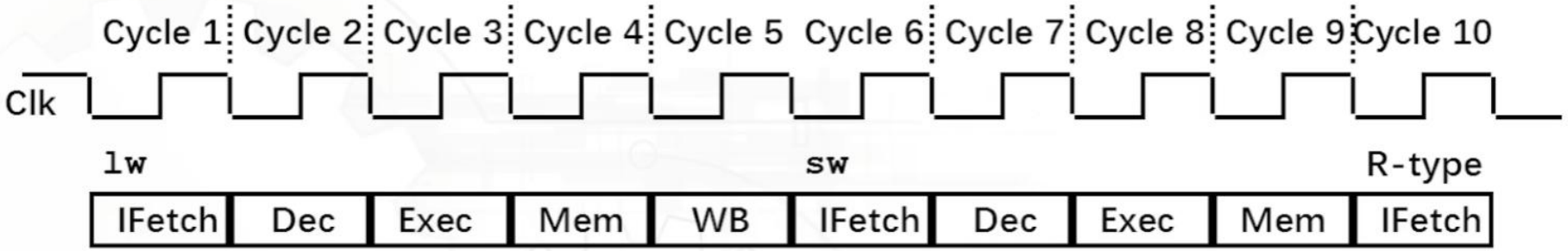


缺点：

1. 时钟周期受限于最慢的一条指令（ slowest instruction ）
2. 芯片面积的浪费
   某些功能部件（例如：加法器）在一条指令周期内只能使用一次，同一个部件需要多个。

优点： 简单、容易理解

解决方案1：多周期完成一条指令



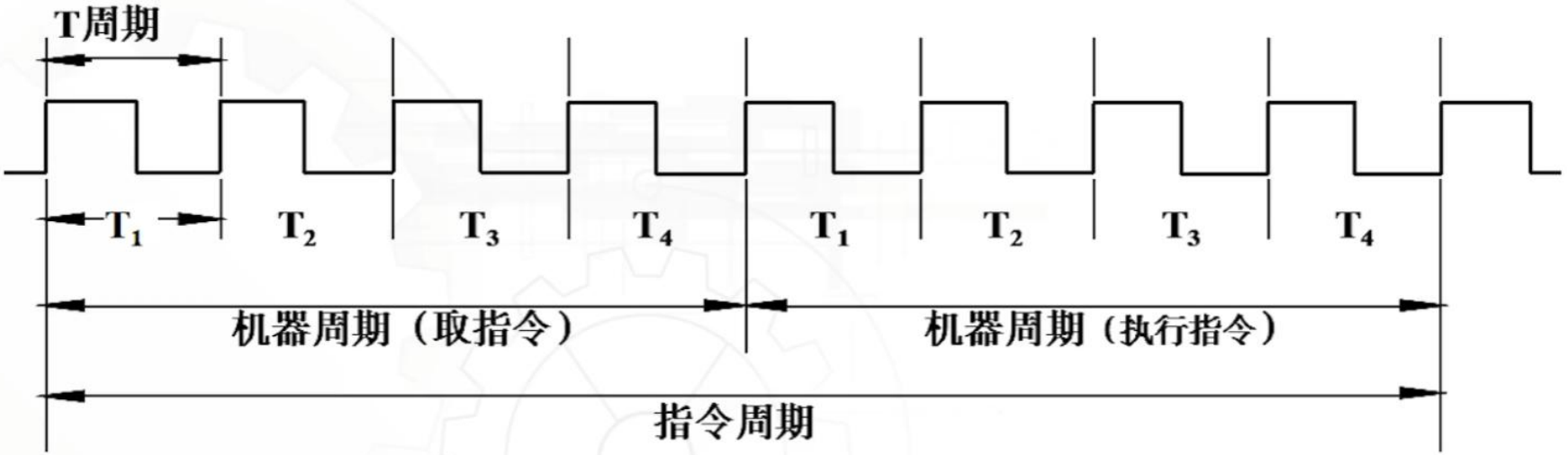
每条指令所花的时钟周期不相同

- 时钟频率变快
- 不同指令所需的时钟周期不同

一个部件在不同的周期可以重复使用

- 不需要增加新部件

指令周期的基本概念



时钟周期：T，节拍脉冲

机器周期，CPU 周期：从内存读出一条指令的最短时间

指令周期：从内存取一条指令并执行该指令所用的时间。由若干个CPU周期组成。一个CPU周期又包含若干个时钟周期（节拍脉冲）。

多周期微架构

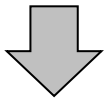
- 目标：让每条指令（接近地）只需要它真正需要的时间
- 思想
  - 独立于指令处理时间来确定时钟周期时间
  - 每条指令所需的时钟周期数由它自己决定
    - 每条指令有多个状态转换
    - 每条指令后续的状态不同

回顾：“处理指令”步骤

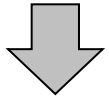
- **ISA 抽象地指定了给定指令和 AS 时，AS' 应该是什么**
  - 它定义了一个抽象的有限状态机，其中
    - 状态 = 程序员可见的状态
    - 下一个状态逻辑 = 指令执行规范
  - 从 ISA 的角度来看，在指令执行过程中，AS 和 AS' 之间没有“中间状态”
    - 每条指令一个状态转换
- **微架构 实现了 AS 如何转换为 AS'**
  - 实现中有很多选择
  - 每条指令多个状态转换
    - 选择1：AS → AS'（在一个时钟周期内将 AS 转换为 AS'）
    - 选择2：AS → AS+MS1 → AS+MS2 → AS+MS3 → AS'（通过多个时钟周期将 AS 转换为 AS'）

多周期微架构

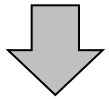
AS = 架构（程序员可见）状态，在指令开始时



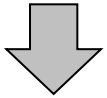
步骤1：在一个时钟周期内处理指令的一部分



步骤2：在下一个时钟周期内处理指令的一部分



…



AS‘ = 架构（程序员可见）状态，在时钟周期结束时

多周期设计的好处

- 关键路径设计
  - 可以继续减少关键路径，无论任何指令的最坏情况处理时间如何。
- 基础（常见情况）设计
  - 可以优化执行“重要”指令所需的状态数量，这些指令占用了大部分执行时间。
- 平衡设计
  - 不需要提供比实际需要更多的能力或资源
    - 需要多次使用资源X的指令，不需要实现多个X
    - 导致更高效的硬件：可以重复使用在指令中多次需要的硬件组件

回顾：性能分析

- 指令的执行时间
  - {CPI} x {时钟周期时间}
- 程序的执行时间
  - 对所有指令求和 [{CPI} x {时钟周期时间}]
  - {指令数} x {平均CPI} x {时钟周期时间}
- 单周期微架构性能
  - CPI = 1
  - 时钟周期时间 = 长
- 多周期微架构性能
  - CPI = 每条指令不同
    - 平均CPI → 希望尽可能小
  - 时钟周期时间 = 短

- 现在，我们有两个自由度可以独立优化

今天的内容

- 微架构—多周期CPU设计
  - LC-3b

我们如何实现多周期CPU ？

- Maurice Wilkes, “The Best Way to Design an Automatic Calculating Machine,” Manchester Univ. Computer Inaugural Conf., 1951.
- 微代码/微程序化机器的概念

THE BEST WAY TO DESIGN AN AUTOMATIC CALCULATING MACHINE

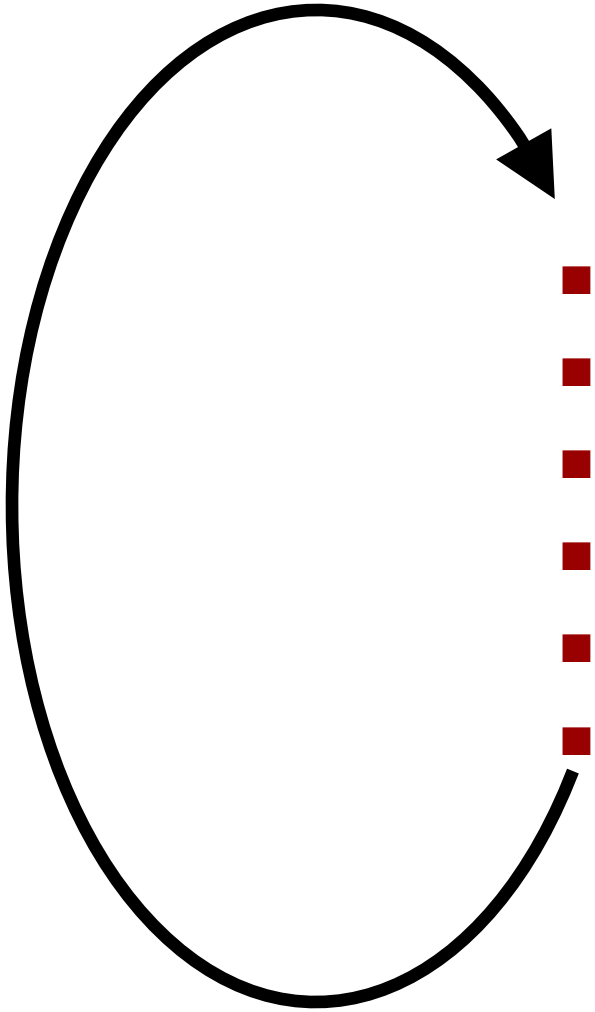
By M. V. Wilkes, M.A., Ph.D., F.R.A.S.



微程序化多周期架构

- **实现的关键思想：**
  - 可以将“处理指令”步骤实现为一个有限状态机，该机器在不同状态之间切换，最终返回到“取指令”状态。
  - 一个状态由在其中激活的控制信号来定义。
  - 下一个状态的控制信号由当前状态决定。

指令处理周期



- 获取指令（Fetch）
- 解码（Decode）
- 地址计算（Evaluate Address）
- 获取操作数（Fetch Operands）
- 执行（Execute）
- 存储结果（Store Result）

基本的多周期微架构

- **指令处理周期划分为“状态”**
  - 指令处理周期的一个阶段可以包含多个状态
- **一个多周期微架构在状态之间切换以处理指令**
  - 机器在某一状态下的行为完全由该状态中的控制信号决定
- **整个处理器的行为完全由一个有限状态机来指定**
- **在一个状态（时钟周期）中，控制信号控制两件事：**
  - 数据路径如何处理数据
  - 如何生成下一个时钟周期的控制信号

微程序化控制(Microprogrammed Control)术语

- 与当前状态相关的控制信号
  - 微指令（Microinstruction）
- 从一个状态过渡到另一个状态的过程
  - 确定下一个状态以及下一个状态的微指令
  - 微序列化（Microsequencing）
- 控制存储器（Control store）存储每个可能状态的控制信号
  - 存储整个有限状态机（FSM）的微指令
- 微序列发生器（Microsequencer）确定将在下一个时钟周期使用的控制信号集合（即，下一个状态）

一个时钟周期内发生了什么？

- 当前状态的控制信号（微指令）控制两件事：
  - 数据路径中的处理
  - 为下一个周期生成控制信号（微指令）

- 数据路径和微序列发生器并行操作

一个时钟周期内发生了什么？

- **当前状态的控制信号（微指令）控制两件事：**
  - 数据路径中的处理
  - 为下一个周期生成控制信号（微指令）
- **数据路径和微序列发生器并行操作**
- **问题：为什么不在当前周期内为当前周期生成控制信号？**
  - 这将延长时钟周期

一个时钟周期内发生了什么？

- **当前状态的控制信号（微指令）控制两件事：**
  - 数据路径中的处理
  - 为下一个周期生成控制信号（微指令）

- **数据路径和微序列发生器并行操作**

- **问题：为什么不在当前周期内为当前周期生成控制信号？**
  - 这将延长时钟周期
  - 为什么会延长时钟周期？

一个简单的LC-3b控制和数据路径

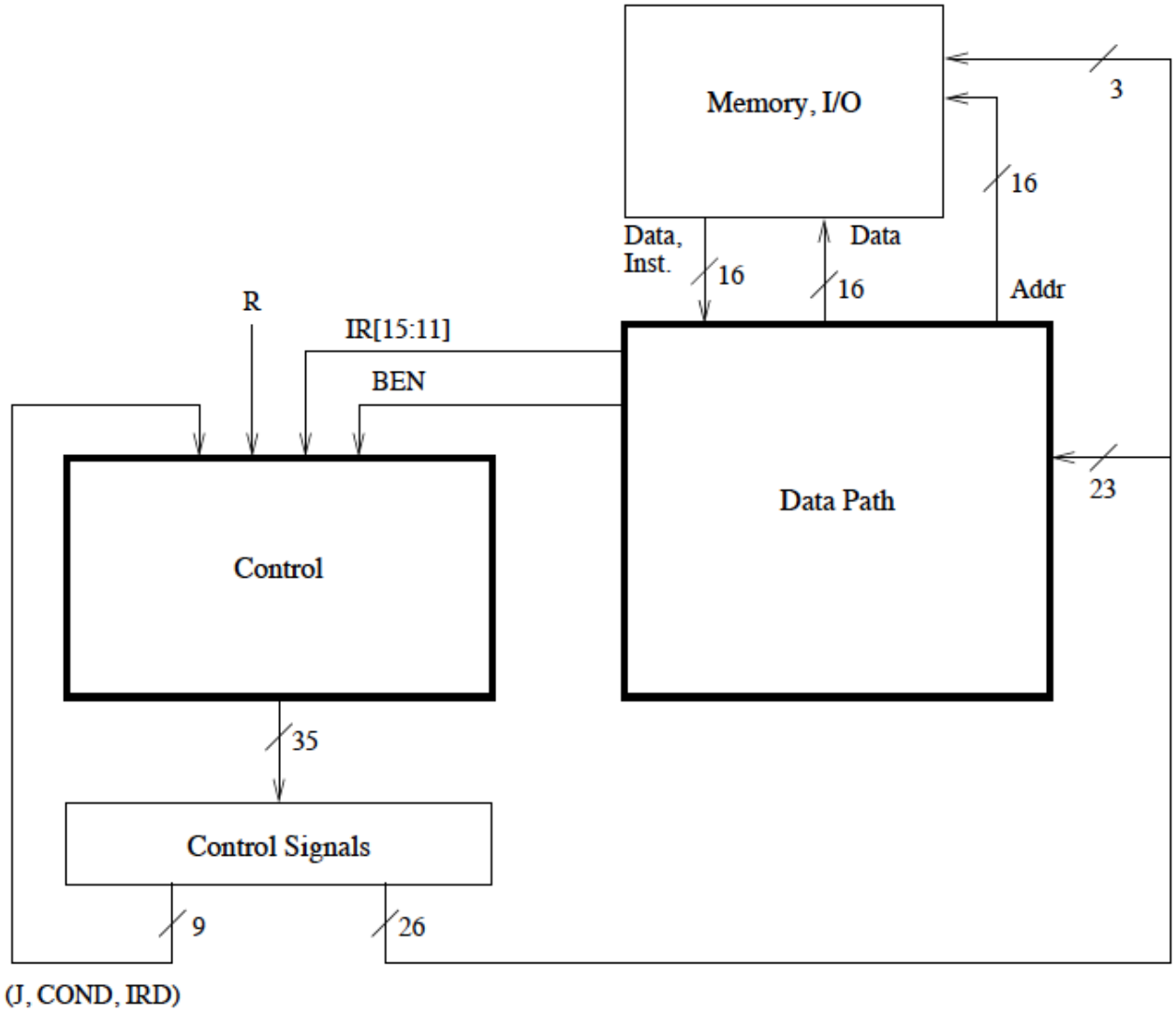


Figure C.1: Microarchitecture of the LC-3b, major components

下节课

- 流水线